

bool datatype:

We can use this datatype to represent boolean values

The allowed values for boolean data type are : True, False

is eligible -> Yes (True)

is not eligible --> No (False)

Is customer active --> yes or no --> True or False

a = True

status = True

is_eligible = True

is_active = True

internally **True** means 1 and **False** means 0

string datatype :

str is the representation of string datatype

A string is sequence of characters, enclosed within a single quotes or double quotes also

it can have a alphanumeric values as well

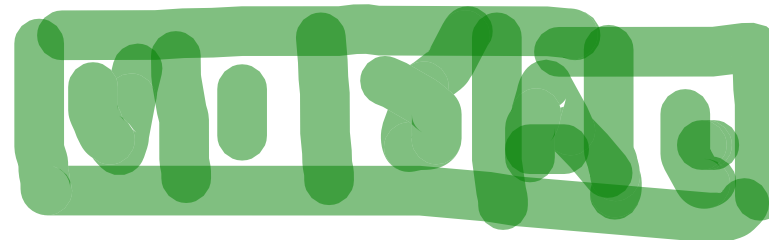
favourite_cricketer = "virat"

customer_name = "dhoni MS"

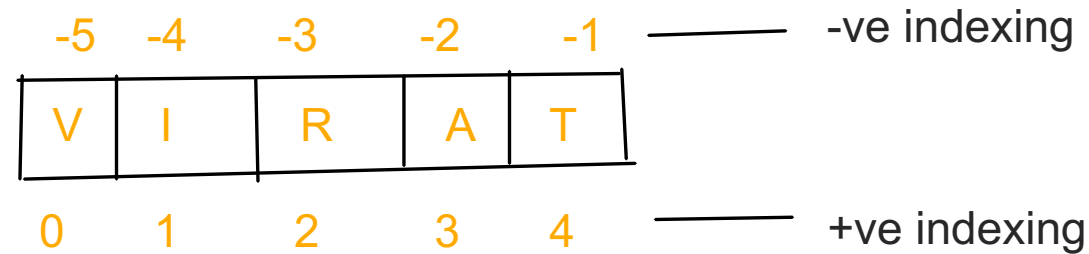
order_info = "ordered selected"

email_id = "test@gmail.com"

"Python class is very and useful" for java students also"



cricketer= "virat"



Slicing of Strings:

slice means a piece of big portion

[] is its slicing operator, which can be used to retrieve parts of string

In python string follows indexing mechanism to store characters

Index always starts from zero (0,1,2,3,4,.....)

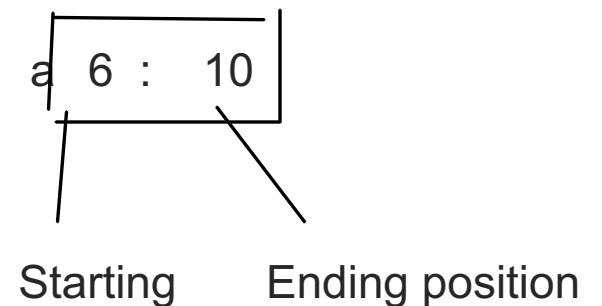
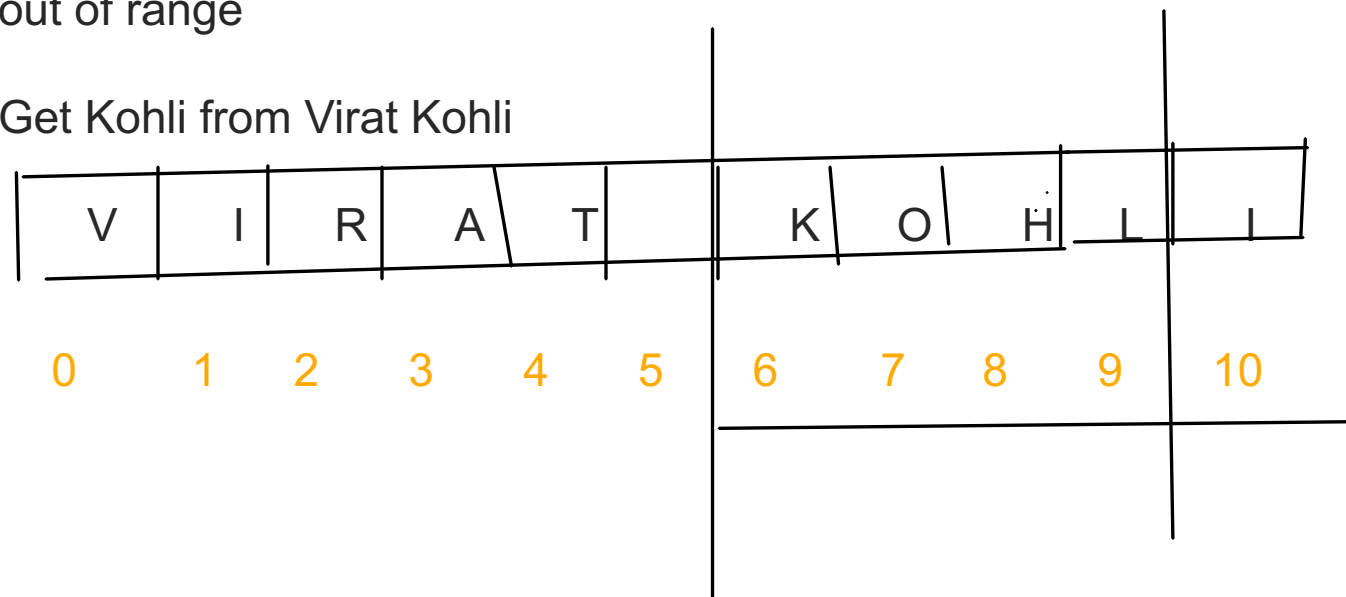
In python indexes can be positive or negative

+ve index means forward direction from left to right

-ve index means backward direction from right to left

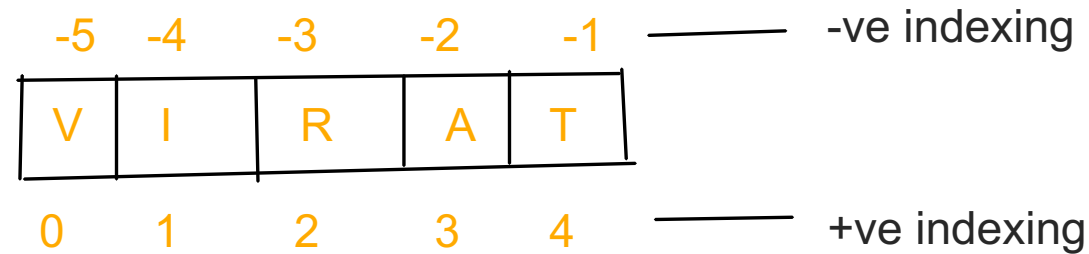
Note: if we try to fetch value using the index more than the indexed position then we get IndexError: string out of range

Get Kohli from Virat Kohli



Starting is position where you want to cut the piece and it includes that position character
end position is where you want to end the piece but ending index **character won't be inclusive**
If we don't give any end position, ending index will be treated as the end of entire string

cricketer= "virat"



Slicing of Strings:

slice means a piece of big portion

[] is its slicing operator, which can be used to retrieve parts of string

In python string follows indexing mechanism to store characters

Index always starts from zero (0,1,2,3,4,.....)

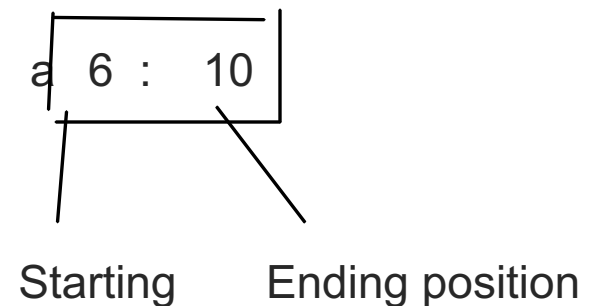
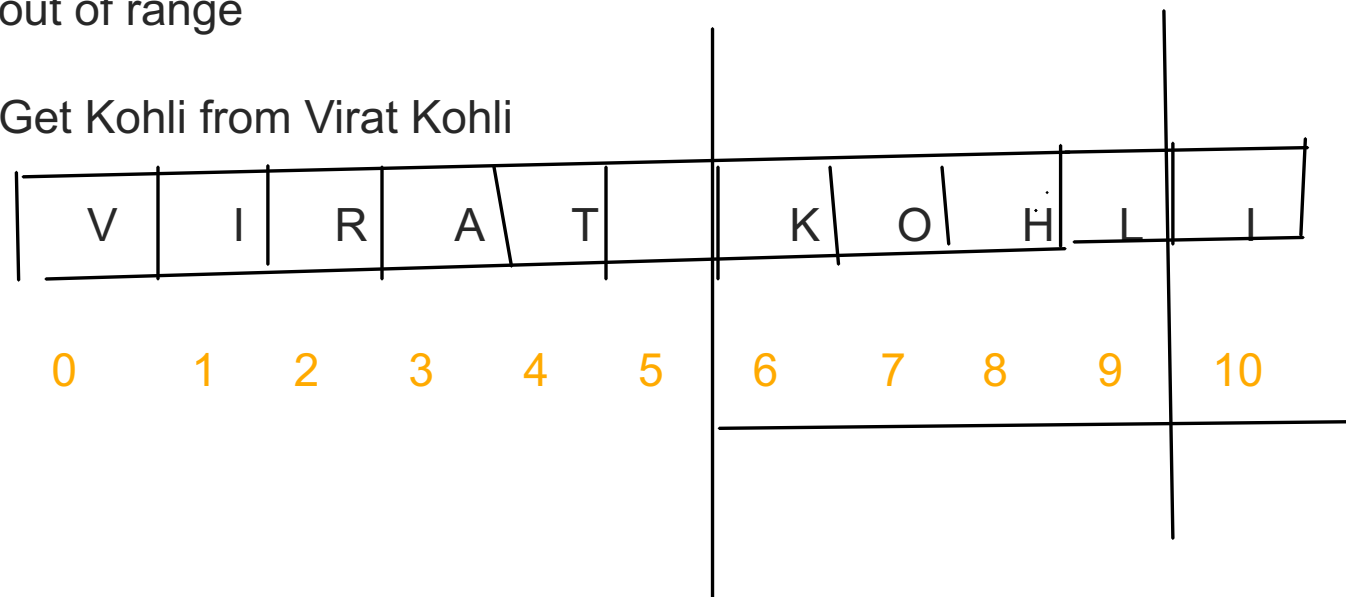
In python indexes can be positive or negative

+ve index means forward direction from left to right

-ve index means backward direction from right to left

Note: if we try to fetch value using the index more than the indexed position then we get IndexError: string out of range

Get Kohli from Virat Kohli



Starting is position where you want to cut the piece and it includes that position character
end position is where you want to end the piece but ending index **character won't be inclusive**
If we don't give any end position, ending index will be treated as the end of entire string

Note:

In python int, float, complex, bool and str are the fundamental datatypes

TYPE CASTING:

a = "10"

b = "20"

The process of converting one type to another type is nothing but a typecasting or type coercion.

These are the following functions for typecasting

1. int() :

We can use this function to convert values from other type to int

Note:

1. We can convert any type to int except complex type

2. If we want to convert string to int type, mandatorily string should have only integral values and that should have base-10, otherwise it will get ValueError

2. float() --> We can use float function to convert other types(int, bool, str, etcc) to float type

Note:

1. We can convert any type to float except complex type

2. If we want to convert string to float type, mandatorily string should have only floating or integral values and that should have base-10, otherwise it will get ValueError

3. complex() --> We can use this function to convert any other type to complex type

4. bool() --> We can use this function convert any toher type to bool

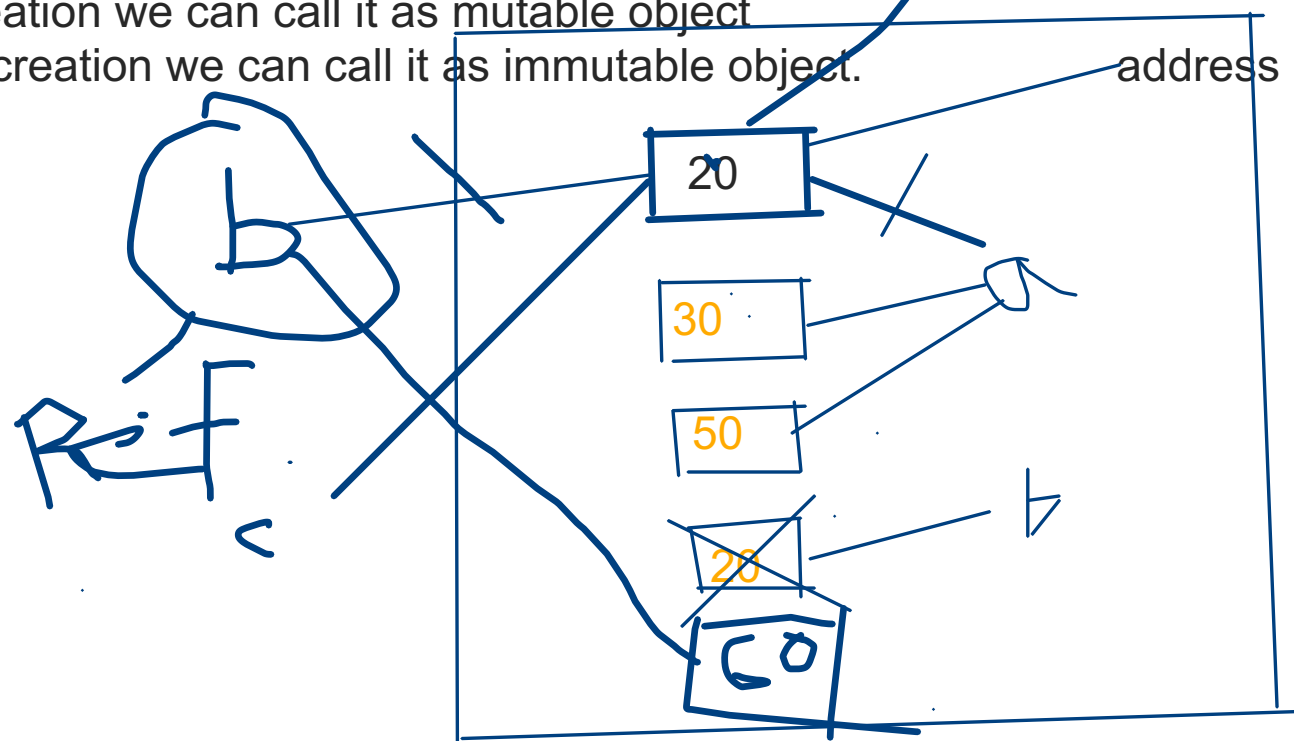
5. str() --> We use this function to convert any other type to string

Immutability w.r.to Fundamental DataTypes:

=====

If I'm able to change the object after creation we can call it as mutable object

If I'm unable to change the object after creation we can call it as immutable object.



In python if a new object is required, PVM wont create new object immediately rather it will check the exist object, if any object has same content or data, it will reuse that object.

If existing objects are not having same content, then only it will create a new object

Advanced Datatype or datastructures:

bytes
bytearray
list
tuple
set
range
dict

bytes datatype :

bytes data type is helpful to represent a group of numbers, like a an array

the only allowed values are 0 to 255, accidentally if we give more than this range then we will get valueerror

Syntax:

we have to use bytes(x) --> x is having sequence of numbers

Ex: bytes([10,20,30,40])

bytes is immutable we can not change its values or reassign once it is created, if we try to modify we will get type error

if we want to store data within 0 to 255 and i dont want anyone to be able to modify/change after creation, then bytes is right choice

1, 2 3 4 5 6 7 8 9, 10, 23 --> state codes in vodafone

bytearray data type:

bytearray is exactly same as bytes datatype except that it is mutable(i.e we can change the values)

the only different between byte and bytearray is bytes is immutable and bytearray is mutable

list datatype:

If we want to store group of values in a single variable, where values having different type of data or same type of data

marks = [10,20,30,40] --> Same type of data, this is possible (homegenious data)

student_data = [12345, "kasi yedumati", 98, True] --> Different type of data(heterogenous data)

1. It maintains insertion order
 2. it allows heterogenous(different type of values) data
 3. duplicate data also allowed
 4. it is growable in nature(we can add data after creation or remove data or modify data or fetch data)
 5. we should create list variable values using []
- Ex: sutdent_list = ["kasi", "virat","dhoni"]

student_names = ["kasi", "bhuvan", "chandu", "varshith", "harika", "chandu"]

0 1 2 3 4 5

student_name[3] --> varshith

0 1 2 3 4 5 6 7

Note: List is an ordered, mutable, heterogenous collection of elements and also duplicated are ordered

tuple data type:

tuple is exactly same as list, except that it is immutable i.e we cannot change the values once it is created

syntax : we have to create a tuple using paranthesis ()

Ex: (10,20,30)

0 1 2

Note: tuple is the read only version of list

range datatype:

range datatype represents sequence of numbers

range is also immutable, the values/numbers present range datatype are not modifiable.

Syntax1:

range(x) --> x will be a number

range(20) --> it generates numbers from 0 to 19

Syntax 2:

range(start, end) --> start will be a number and end also should be a number

Syntax3:

range(start, end, increment)

start --> it is where i want to sequence range should start from

end --> it is where i want to stop range

increment --> how many values you have to increment for every generate sequence numbers

range(10, 20, 2)

Set datatype:

Whenever we want to store group of values where insertion order is not required and duplicated are not allowed, in that case we go for set

1. insertion order is not preserved
2. duplicates are not allowed
3. heterogeneous objects are allowed (same or different type of values)
4. it doesn't follow indexing
5. it follows hashing mechanism to insert or update data or manage data in set
6. it is mutable (once the object is created we can modify)
7. it is growable in nature

```
fruits_set = {'banana', 'apple'}  
fruits_set = set() --> create an empty set
```

frozenset Datatype:

frozenset is exactly same as set, except that it is immutable.

Once it is created, we can't add, remove or modify its items.

Syntax:

```
frozenset([10,20,30])
```

dict datatype :

If we want to store a group of values as **key-value** pairs then we should go for dictionary datatype

Syntax:

```
contacts = {"name" : "mobilenumber"}
```

```
employee_data = ["kasi", 1234, 123000, "hyd"] --> list data form
```

```
employee_data =
```

```
{  
    "name" : "kasi",  
    "employeeid" : 12345,  
    "salary" : 12345.50,  
    "location" : "hyd"  
}
```

Key must be unique

value can be unique or duplicate

immutable datatype (string, tuple, frozenset)

Using key we can access the data

Values can be any datatype

if we try to add same key twice, it will just consider latest entry value or it overrides values within the same key
it won't throw any error

Datatype	Duplicates	Insertion order	heterogenous	mutable or immutable	syntax	follows index?
int	not applicable	not applicable	only integers 0 to 9	immutable	a = 10; b = 20	
float	not applicates	not applicable	only floating values	immutable	a= 10.5	
complex	not applicable	not applicable	real and imaginary	immutable	a = 3+0j	
bool	not applicable	not applicable	True, False	immutable	a = True ; a= False	
string	allowed	applicable	No	immutable	a= "hello"	Yes, eg: a[2]
bytes	allowed	applicable	No(0 to 255)	immutable	b = bytes([10,20,30])	Yes
bytearray	allowed	applicable	No	mutable	b =bytearray([10,20])	Yes
list	allowed	applicable	Yes	mutable	names=['shreyas','sai']	Yes, eg: names[1]
tuple	allowed	applicable	Yes	Immutable	x=(1,3+2j, 'chandu')	Yes, eg: x[2][2]
range	allowed	applicable	No	mutable	x=range(0,9); 0,1,2,3 4,5,6,7,8	Yes
set	not allowed	not preserved	Yes	mutable	x={1,2,3,4}	No (because it follows hashing)
frozenset	not allowed	not preserved	Yes	Immutable	x=frozenset([1,2,3])	no
dict	duplicate keys are not allowed	applicables	Yes	Mutable	d={'a' : 1, 'b':2}	we access values with key

Operators:

Operator is a symbol or keyword using which we can perform operations

- 1. Arithmetic operators
 - + -> Additionsum values
 - -> subtraction
 - * -> multiplication
 - / -> Division operator
 - % -> module operator (it returns the reminder)
 - // -> Floor division operator
 - ** -> Power operator or exponential operator
- + operator we can against strings also, we call it as concatination operator
- * operator also we can use against strings, but atleast one value should be integer
- String multiplication operator

- 2. Relational or Comparision operators
 - >
 - <
 - >=
 - <=

Comparison operators will compare any two values and return boolean values True or False
Equality operators -> == !=

- 3. Logical operators
There are three logical operators: **and**, **or**, **not**
These keywords/operators we can apply on all datatypes

These operators also check and return **boolean value**

and -> If both arguments are **True** then only result is **True**
or -> If atleast one argument is **True** then output will be **True**
not -> **complement** (if we apply **not** for **True** it will become **False**, if we apply **not** for **False** it will return **True**)

	A	D	
and	True True False	True False False	True False False

	A	B	Result
or	True True False False	True False True False	True True True False

0 means False
non-zero means True
empty string is always treated as False

If a evaluates to False return a otherwise it returns y

- 4. Assignment operators
We can use assignment operator to assign value to a variable
Eg: x = 10
We can combine assignment operator with mathematical operators

x += 10

- 5. Ternary Operators
Syntax:
x = **firstValue** if **condition** else **secondValue**

If condition becomes True then firstValue will be returned, else(if condition becomes False) secondValue will be returned

- 5. Bitwise operators
6. Special operators
Python has two special category operators they are
6.1 : Identity Operators
We can use identity operators for address comparison
is
is not

Eg: a = 20
b = 30
a is b -> This returns **True** if both a and b are pointing to the same object, otherwise returns **False**
a is not b -> This returns **True** if both a and b are **not pointing to same obj**, otherwise returns **False**

Note: We can use is operator for address comparison whereas == operator for content comparison

- 6.2 : Membership operators:
We can use membership operators to check whether the given object is present in the collection or datastructures (String, List, Set, Tuple or Dict)

in -> Returns True if the given obj present in the specified collection
not in -> Returns True if the given object is not present in the collection

Bitwise Operators:
We can apply these operators bitwise
These operators we can apply only on int and boolean types, if we try to apply on any other type we will get Error

Eg: print(5&4) -> Valid
print(True & True) -> Valid
print("abc" & "xyz") -> Invalid (Error)
print(0.5&10.5) -> Invalid(Error)

5 -> 0101
4 -> 0100
15 8 4 2 1
2 2 2 2
0 1 0 0 -> 0+0+4+0=4

4 - 0100
5 - 0101
15 8 4 2 1
0 0 0 0 -> 0+4+0+0=4
1 - 0101 -> 5
6 - 0110
8 - 1000
8 - 0000
1 - 1110 -> 8+4+2+0=14

4 0100
5 0101
6 0011

bitwise compliment operators:
We have to apply complement for total bits(single value)

Eg: print(-5) -> -6

Operator	Description
&	If both bits are 1 then only result is 1 otherwise result is 0
	If atleast one bit is 1 then result is 1 otherwise result is 0
^	If bits are different then result is 1 otherwise result is 0
~	bitwise complement operator i.e 1 means 0 and 0 means 1
>>	Bitwise right shift operator -> it shifts positions to the right, and result the final value
<<	bitwise left shift operator -> it shifts positions to the right and result the final value

<< left shift operators
Eg: print(10<<2)
10 -> 40

right shift >>
print(10>>2)
10 -> 2

Python Comments:

Comments in python are used to explain the code and provide futuristic TODO items
these comments are ignore by python during program execution

Advantages:

- 1. Readability improvement
- 2. It can be used to identity the funcitonality or structure of the code

Single line comments :

Single line comments are starts with hashtag #

Ex:

this is a single comment

Multi-line comments:

We can apply hashtags multiple times
using a string literals using `"""comment"""`
`"""testcomment"""`

Input and Output Statements:

Static inputs/values --> If the value or input is always same, we call it as static value/input

Dynamic inputs/Values --> If the value or input keeps changing over the time, we call it as dynamic input

Whenever we are building applications or command line based scripts or automation scripts, we take the input from Either ui or command prompt or terminal

To take the dynamic inputs from the keyboard we have a function called **input()**

input():

it is used to take dynamic input from keyboard while running the program, and it will wait for user to enter the input
and start executing the program once the data is entered

input function always return data in string type, before perfroming any operation we have to do the typecasting if at all it is needed.

reading multiple values at a time
split() is the function in string, which will split values based on delimiter/saperator, if we dont specify any saperator
by default it will saperate based on space(" ") and it will return splitted values in list type
a = input("Enter 2 numbers in a space saperated:").split()

eval():

It will take a string as in put and evaluate the result

eval can evaluate the input to list, tuple, set, based on the provided input

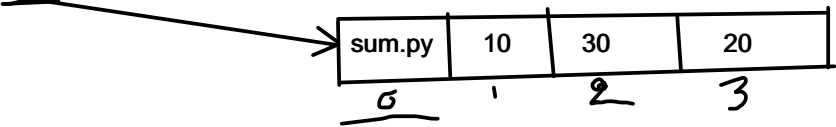
Command Line Arguments:

The argument which are passing at the time of execution are called command line arguments

Eg: python3 sum.py 10 30 20

Command line arugments

Within in the python program these arguments are available in **argv**, which is present in **sys** module
argv --> is an internal python manged parameter which of list type



Note1: By default all arguments will be treated as strings only

Note2: Within the python program command line args are available in string form based on our requirement, we need to convert them to required type

Note3: Usually space is saperator between command line args, if our command line arg itself has a space then we should enclose within the double quotes

Output statements:

We can use print() function to display output

Form-1: print() without any argument

Print Syntax:

```
print(*args, sep= ' ', end='\n', file=sys.stdout, flush= False)
```

Form3: print() with variable number of arguments

```
a,b,c= 10,20,30  
print("numbers are", a,b,c)
```

by default sep is space, we can override by explicitly giving a separator

```
print(a,b,c, sep='&')
```

end parameter is used to end the current print statement, by default end will be \n which is new line

Form5: print with objects

Form6: print(string, variable list)

we can use print statement with string and any number of arguments

Form7: print(formatted string)

%i --> int value

%d --> int value

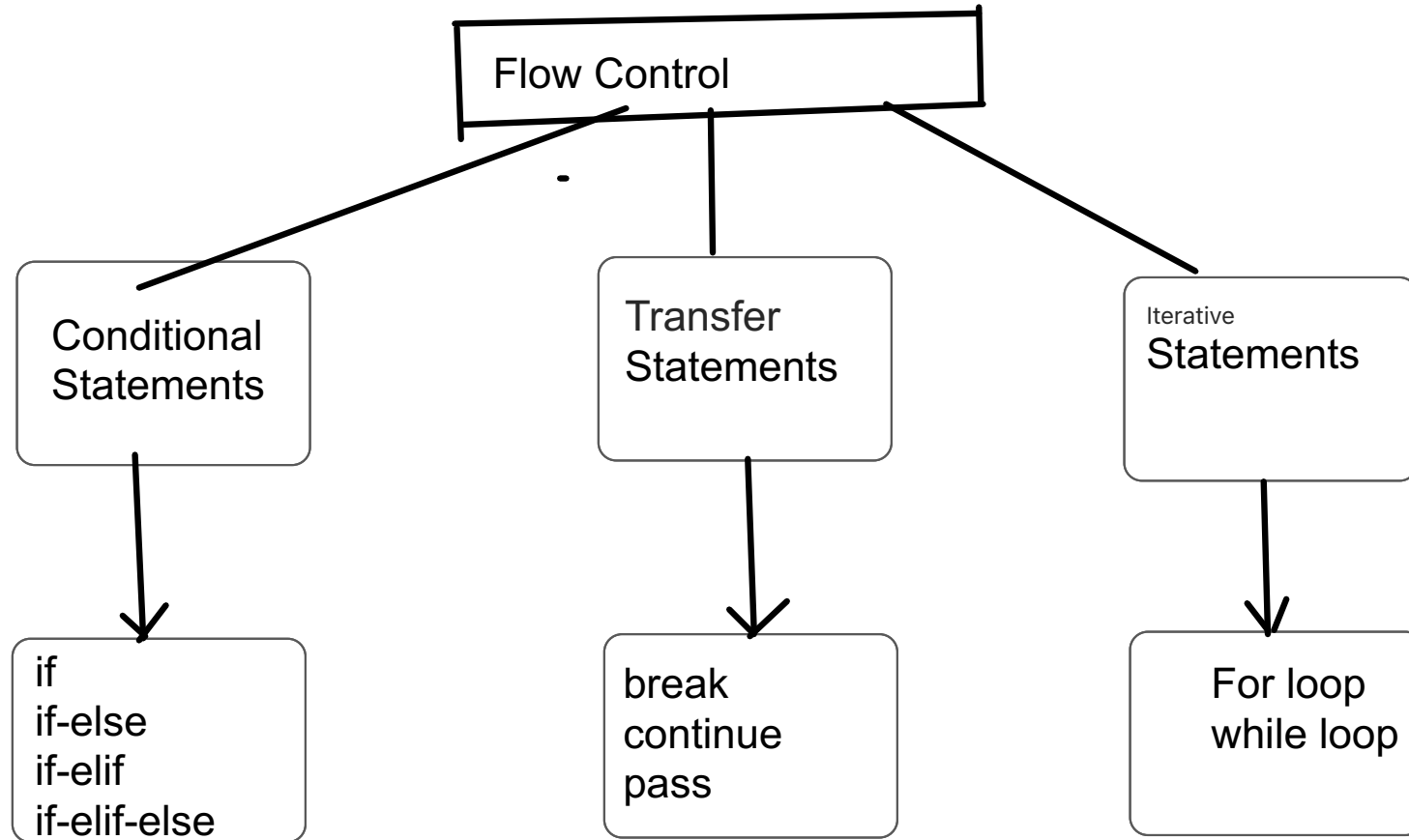
%f --> float

%s --> string

Form8: print with replacement operator { }

Flow Control:

Flow control describes the order in which statements should be executed and also which statements to execute



Conditional statements:

Conditional statements are such that, if your condition is met only then the statements will be executed.
aka, if we want to execute any statement(s) conditionally, then we go for conditional statement

if:

Syntax:

statement1

statement2

if condition:
 statement3
 statement4

statement5

statement6

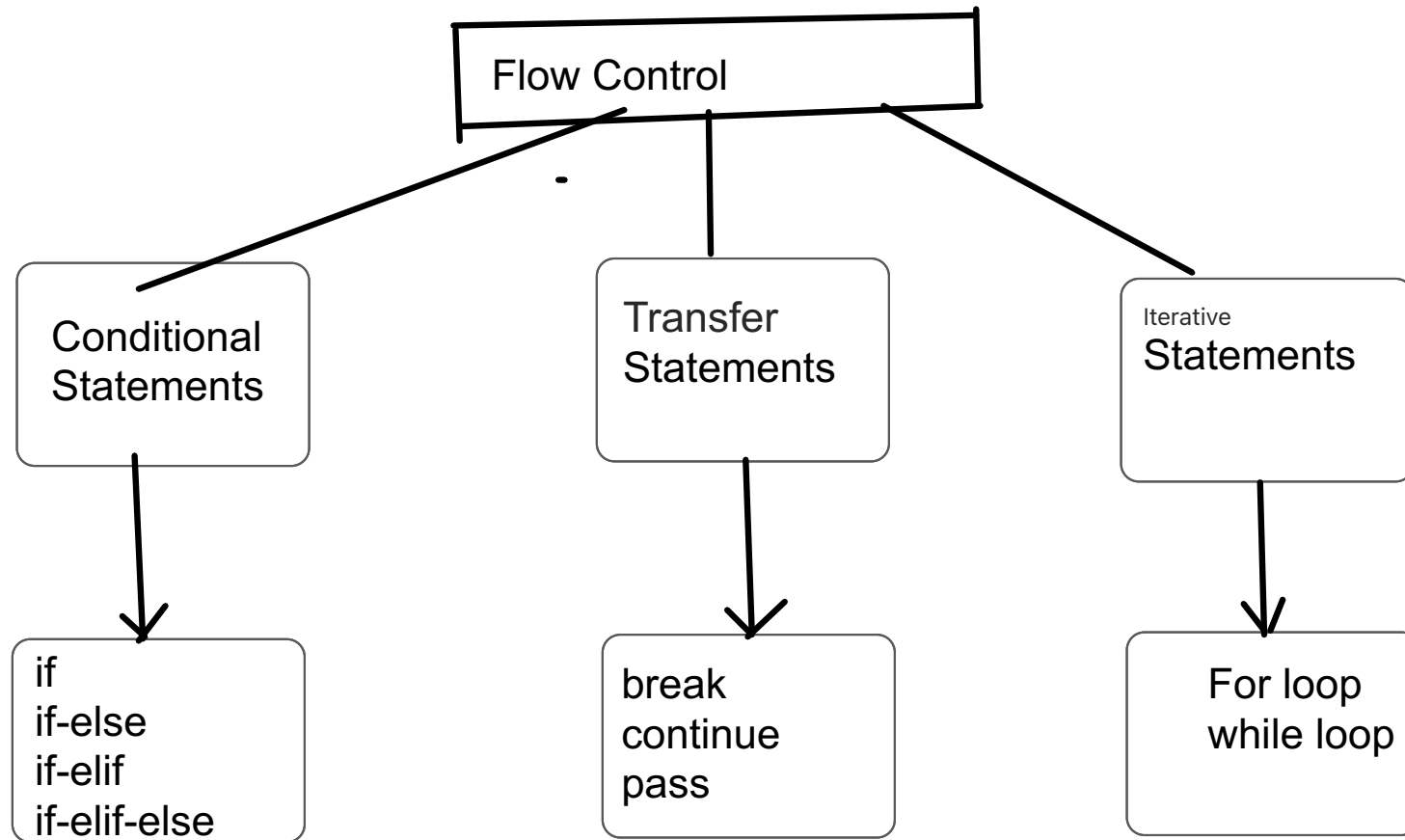
in this program statement3, 4 will be execution only when condition is True
statement1,2,5,6 will execute, irrespective of condition is True or False

if block of statements

Note: At a time either if block statements or else block statements will execute not both

Flow Control:

Flow control describes the order in which statements should be executed and also which statements to execute



Conditional statements:

Conditional statements are such that, if your condition is met only then the statements will be executed.
aka, if we want to execute any statement(s) conditionally, then we go for conditional statement

if:

Syntax:

statement1

statement2

if condition:
 statement3
 statement4

statement5

statement6

in this program statement3, 4 will be execution only when condition is True
statement1,2,5,6 will execute, irrespective of condition is True or False

if block of statements

Note: At a time either if block statements or else block statements will execute not both

3. if-elif-else:

=====

Syntax:

if condition1:

 Action1

elif condition2:

 action2

elif condition3:

 action3

else:

 action4

At a time only one action will/has to be executed, then we got if-elif-else

condition1 is True --> Action1 will be executed

condition2 is True -> action2 will be executed

condition3 is True --> action3 will be executed

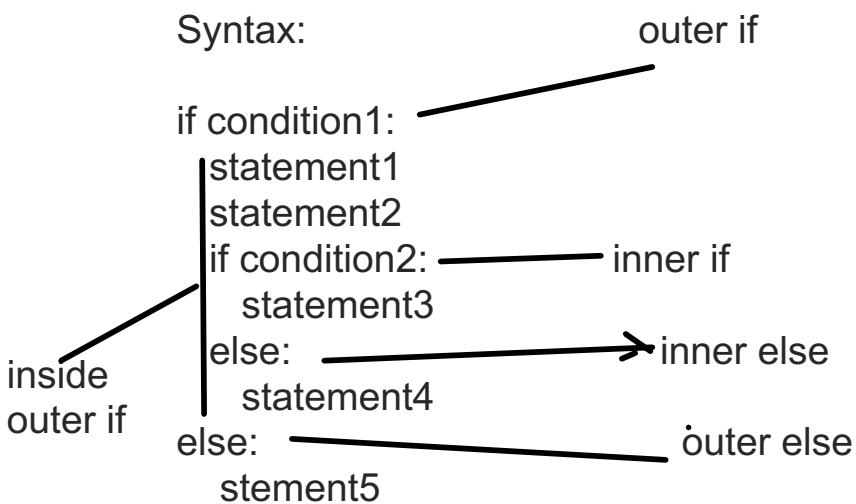
if no condition is True then else block will execute

we don't have switch cases in python unlike other programming languages

nested if :

a if within another if or if withing another else

Syntax:



if condition1 is True all the statements including nested if-else will execute

if condition1 and nested if condition 2 is True then only statement3 will execute

Iterative/repetitive Statements:

If we want to execute a group of statements multiple times then we should go for iterative statements

Python supports 2 types of iteratives

1. for loop
2. while loop

1. for loop:

If we want to execute some action for every element present in the some sequence(string, list, tuple, dict, range, set, frozenset, etc...)

Syntax:

```
for x in sequence:  
    body(statements)
```

sequence --> string or any collection(list, set, tuple, etc....)

x means each value in the sequence

body will be executed for every element in the given string

2) while loop:

If we want to execute a group of statements iteratively/repetitively until some condition is false, then we should go

while loop

Syntax:

```
while condition:  
    body
```

group of statments that we want to execute when condition is True

Nested loops:

Sometimes we can take a loop inside another loop, which we call it as nested loops.

```
for i in range(1,5):  outer for loop      lter1: i =1, j= 1 2 3 4  
    for j in range(1,5): --> inner for loop  lter2: i=2, j = 1 2 3 4  
        print(j)      --> inner for loop body lter3: i =3, j = 1 2 3 4  
                                lter4: i =4, j = 1 2 3 4  
                                lter5: end the loop  
outer for loop body
```

Transfer statements:

1) break

We can use break statement inside loops to break the loop execution based on some condition

2) continue:

we can use continue statement to skip current iteration and continue the next iteration

3) pass:

pass is a keyword in python

In our python programming syntactically, if block is requires a block of statements.

but if we want to give empty block python will throw en error, using pass keyword we can create empty blocks

pass -> it is an empty statement

--> it is null statement

del statements:

del is a keyword in python

We can delete variables and objects which are pointing to mutalble objects, but we cannot delete immutable objects

```
name = "kasi"
```

```
del name --> valid
```

```
del name[0] --> not valid becuase "kasi" is immutable object
```